

1

**Wordplay**

2

Given an integer  $N$  where  $N$  denotes the total number of words in `word_list`. You are given an array `word_list` of words that can be used to make the longest word without any repeating characters. Return the length of the longest word that can be formed by concatenating the words from the array `word_list` without any repeating characters.

3

**Function description**

Complete the `solve()` function. This function takes the following 2 arguments and returns the answer:

- $N$ : Represents the total number of words in `word_list`
- `word_list[]`: Represents the array `word_list` that consists of the words that can be used to make the longest word without any repeating character

**Input format for custom testing**

**Note:** Use this input format if you are testing against custom input or writing code in a language where we don't provide boilerplate code

- The first line contains an integer  $N$  denoting the total number of words in `word_list`.
- The second line contains an array of  $N$  strings denoting the words that can be used to make the longest word.

**Output format**

```
#include <iostream>
using namespace std;
int solve (int N, char** word_list) {
    // Write your code here
    return 0;
}
```



words that can be used to make the longest word.

### Output format

Return the length of the longest word that can be formed by concatenating the words from the array *word\_list* without any repeating characters.

### Constraints

$$1 \leq N \leq 16$$

*word\_list[i]* contains  $[a-z]$

#### Sample input

```
5
ab
cd
ef
ab
de
```

#### Sample output

```
6
```

### Explanation

- ab cd ef ab de
- The longest word that can be formed using the words is ab + cd + ef
- Therefore, the answer is 6 i.e. the length of abcdef.

Solution:

```

#include <bits/stdc++.h>
using namespace std;

bool hasUniqueChars(const string &word) {
    vector<bool> seen(26, false);
    for (char c : word) {
        if (seen[c - 'a']) return false;
        seen[c - 'a'] = true;
    }
    return true;
}

void backtrack(vector<string> &words, int index, int mask, int length, int &maxLen) {
    maxLen = max(maxLen, length);

    for (int i = index; i < words.size(); ++i) {
        int currMask = 0;
        bool valid = true;

        for (char c : words[i]) {
            int bit = c - 'a';
            if ((currMask >> bit) & 1) {
                valid = false; // duplicate in word
                break;
            }
            currMask |= (1 << bit);
        }

        if (!valid || (mask & currMask)) continue; // conflict with previous mask

        backtrack(words, i + 1, mask | currMask, length + words[i].size(), maxLen);
    }
}

int solve(int N, vector<string> &words) {
    vector<string> cleanWords;
    for (const string &word : words) {
        if (hasUniqueChars(word))
            cleanWords.push_back(word);
    }

    int maxLen = 0;
    backtrack(cleanWords, 0, 0, 0, maxLen);
    return maxLen;
}

```



**New game**

Bob wants to complete all the levels of a game in the minimum possible time. Each level takes a consecutive number of minutes to complete, and Bob can only play the game during breaks at work. Each day has a total of minutes, and there is a break that lasts for minutes. Saiki's goal is to determine if he can complete the game and, if so, find the minimum number of minutes required to finish all the levels.

**Function description**

Complete the solve function. This function takes the following 5 parameters and returns the answer:

- $n$ : Represents the number of breaks
- $m$ : Represents the number of game levels
- $t$ : Represents the time to complete each level
- $k$ : Represents the length of each day
- $B$ : Represents the list of pairs with values  $l$  and  $r$  the range of break

**Input format for custom testing**

**Note:** Use this input format if you are testing against custom input or writing code in a language where we don't provide boilerplate code.

- The first line contains  $T$ , which represents the number of test



- $B$ : Represents the list of pairs with values  $l$  and  $r$ .
-  break

### Input format for custom testing

**Note:** Use this input format if you are testing against custom input or writing code in a language where we don't provide boilerplate code.

- The first line contains  $T$ , which represents the number of test cases.
- For each test case:
  - The first line contains 4 integers  $n, m, t, k$  denoting the number of breaks per day, the number of game levels, the time to complete each level, and the length of each day.
  - Each of the next  $n$  lines consists of 2 integers  $l_i$  and  $r_i$ , representing the range of  $i^{\text{th}}$  break.

### Output format

For each test case print in a new line, the minimum number of minutes needed to finish the game, or -1 if Saiki can't complete the game at all.

### Constraints

$$1 \leq T \leq 2 \cdot 10^5$$

$$1 \leq n \leq 10^5$$

$$1 \leq m, k, t \leq 10^9$$

$$0 \leq l_i < r_i \leq k$$

[Previous Question](#)



$$1 \leq t \leq 2 \cdot 10^6$$

$$n \leq 10^5$$

$$1 \leq m, k, t \leq 10^9$$

$$0 \leq l_i < r_i \leq k$$

$$r_i < l_{i+1}$$

The sum of all  $n$  overall the test cases is not greater than  $10^6$ .

#### Sample input

```
2
1 5 3 9
3 7
1 5 5 9
3 7
```

#### Sample output

```
38
-1
```

#### Explanation

The first line represents the number of test cases,  $t = 2$ .

##### The first test case

Saiki should start at the fifth minute of the first day and finish it before the seventh minute of the fifth day so he needs 38 minutes to finish the whole game.

##### The second test case

[Previous Question](#)



1

**Explanation**

The first line represents the number of test cases,  $t = 2$ .

2

**The first test case**

3

Saiki should start at the fifth minute of the first day and finish it before the seventh minute of the fifth day so he needs 38 minutes to finish the whole game.

**The second test case**

Saiki can't finish the game at all; the day has one break of length 4 and each level needs 5 minutes to complete.

**Note:**

Your code must be able to print the sample output from the provided sample input. However, your code is run against multiple hidden test cases. Therefore, your code must pass these hidden test cases to solve the problem statement.

**Limits**

Time Limit: 1.0 sec(s) for each input file

Memory Limit: 256 MB

Source Limit: 1024 KB

**Scoring**

Score is assigned if any testcase passes

**Allowed Languages**

Bash, C, C++14, C++17, Clojure, C#, D, Erlang, F#, Go, Groovy, Haskell, Java 8, Java 14, JavaScript(Node.js), Julia, Kotlin, Lisp (SBCL), Lua, Objective-C, OCaml, Octave, Pascal, Perl, PHP, Python, Python 3, Python 3.8, Racket, Ruby, Rust, Scala, Swift, TypeScript, Visual Basic